Informe Práctica 2: Multiplicación Rápida de Enteros

Asignatura: Análisis y Diseño de Algoritmos

Tema: Divide y Vencerás - Algoritmo de Karatsuba

Integrantes: Juan Daniel Martinez Fernandez, Samuel Posada Londoño

---

Contexto y Objetivo

En la programación tradicional, los tipos de datos nativos como int o long long tienen un límite físico de almacenamiento dictado por la arquitectura del procesador (por ejemplo, 64 bits solo pueden almacenar números de hasta 19 dígitos). Sin embargo, en áreas como la criptografía o el cálculo científico de alta precisión, es indispensable multiplicar números que tienen cientos o miles de dígitos. Para lograr esto, los números deben tratarse como arreglos o cadenas de texto, operando dígito por dígito. El objetivo de esta práctica es comparar el método de multiplicación tradicional frente a un enfoque más avanzado basado en "Divide y Vencerás": el algoritmo de Karatsuba.

1. Deducción de la identidad de Karatsuba

Para entender este algoritmo, primero debemos analizar el método clásico. Si queremos multiplicar dos números enormes, digamos **X** e **Y**, de tamaño n, la forma directa nos obliga a multiplicar cada dígito del primer número por todos los dígitos del segundo. Si aplicamos la técnica de "Divide y Vencerás" de manera ingenua, partiríamos cada número por la mitad. Así, **X** se divide en una mitad "alta" (los dígitos de la izquierda) y una mitad "baja" (los dígitos de la derecha). Lo mismo hacemos con **Y**.

Al hacer la multiplicación algebraica de estas partes divididas, el resultado requiere calcular cuatro productos recursivos independientes:

- La mitad alta de X multiplicada por la mitad alta de Y.
- La mitad alta de X multiplicada por la mitad baja de Y.
- La mitad baja de X multiplicada por la mitad alta de Y.
- La mitad baja de X multiplicada por la mitad baja de Y.

El problema con esto es que seguimos teniendo un crecimiento cuadrático; dividir para terminar haciendo cuatro operaciones casi del mismo

peso no mejora la eficiencia del algoritmo.

La optimización de Karatsuba demuestra que podemos encontrar el resultado haciendo **sólo tres multiplicaciones**. Él propuso calcular lo siguiente:

- **P_1:** El producto de las dos mitades altas.
- **P_2:** El producto de las dos mitades bajas.
- **P_3:** El producto de la suma de las mitades (alta + baja de X) multiplicado por (alta + baja de Y).

Si analizamos algebraicamente el desarrollo de **P_3**, al multiplicar esas sumas obtenemos cuatro términos: el producto de las altas, el producto de las bajas, y los dos productos cruzados. Como ya calculamos las altas (P_1) y las bajas (P_2), simplemente se restan a P_3. Nos da exactamente los dos términos centrales que nos faltaban. Gracias a esto, reducimos la cantidad de llamadas recursivas de cuatro a tres, lo cual genera un impacto enorme en el tiempo de ejecución a medida que los números crecen exponencialmente.

2. Análisis de la recurrencia

Dado que el algoritmo de Karatsuba se llama a sí mismo tres veces reduciendo el tamaño del problema a la mitad en cada paso, la ecuación matemática que modela su tiempo de ejecución es **$T(n) = 3T(n/2) + \Theta(n)$**. El término lineal final, $\Theta(n)$, representa el costo de tiempo que nos toma hacer las sumas, las restas y añadir los ceros que son los desplazamientos para recomponer el número final. Podemos resolver esta recurrencia de dos formas:

Resolución por el Teorema Maestro

Aplicando la fórmula general del Teorema Maestro, identificamos nuestros parámetros:

- **$a = 3$**: El número de subproblemas (las tres multiplicaciones P_1, P_2 y P_3).
- **$b = 2$**: El factor por el cual se reduce el tamaño del problema (partimos a la mitad).
- **$f(n) = \Theta(n)$**: El costo del trabajo adicional fuera de la recursión (sumas y restas).

El siguiente paso es evaluar el costo de las hojas del árbol, que está dado por $n^{\log_b(a)}$, es decir, $n^{\log_2(3)}$. Sabiendo que el logaritmo en base 2 de 3 es aproximadamente 1.585, comparamos $n^{1.585}$ contra nuestro $f(n)$ que es n^1. Como el término exponencial domina abrumadoramente al término lineal, el teorema nos ubica en el "Caso 1". Esto nos permite concluir formalmente que la complejidad asintótica total del algoritmo es **$\Theta(n^{1.585})$**, superando al $\Theta(n^2)$ clásico.

Resolución por el Árbol de Recurrencia

Si visualizamos la ejecución del programa como un árbol genealógico, en la raíz tenemos nuestro problema original de tamaño *n*, el cual nos

cuesta un trabajo proporcional a $c \cdot n$. En el nivel 1, el problema se divide en 3 ramas de tamaño $n/2$. En el nivel 2, tendremos 9 ramas de tamaño $n/4$, y así sucesivamente.

En un nivel genérico , tendremos 3^i subproblemas, cada uno de tamaño $n/2^i$. Si sumamos el trabajo de todo ese nivel, el costo es $c \cdot n(3/2)^i$. Como la fracción 3/2 es mayor a 1, significa que el costo por nivel va aumentando conforme bajamos en el árbol. La recursión se detiene cuando llegamos a los casos base (números de 1 dígito), lo que nos da una altura máxima del árbol de $\log_2(n)$.

Como el mayor peso de las operaciones recae en el último nivel, el costo total está dictado por el número de hojas, que es $3^{\log_2(n)}$. Usando propiedades básicas de los logaritmos, esta expresión se puede reescribir como $n^{\log_2(3)}$. Nuevamente, comprobamos matemáticamente de dónde proviene el exponente y confirmamos la complejidad.

3. Decisiones de diseño y dificultades de implementación

Llevar esta teoria a un lenguaje como C++ presentó varios retos técnicos que resolvimos mediante ciertas desiciones de diseño:

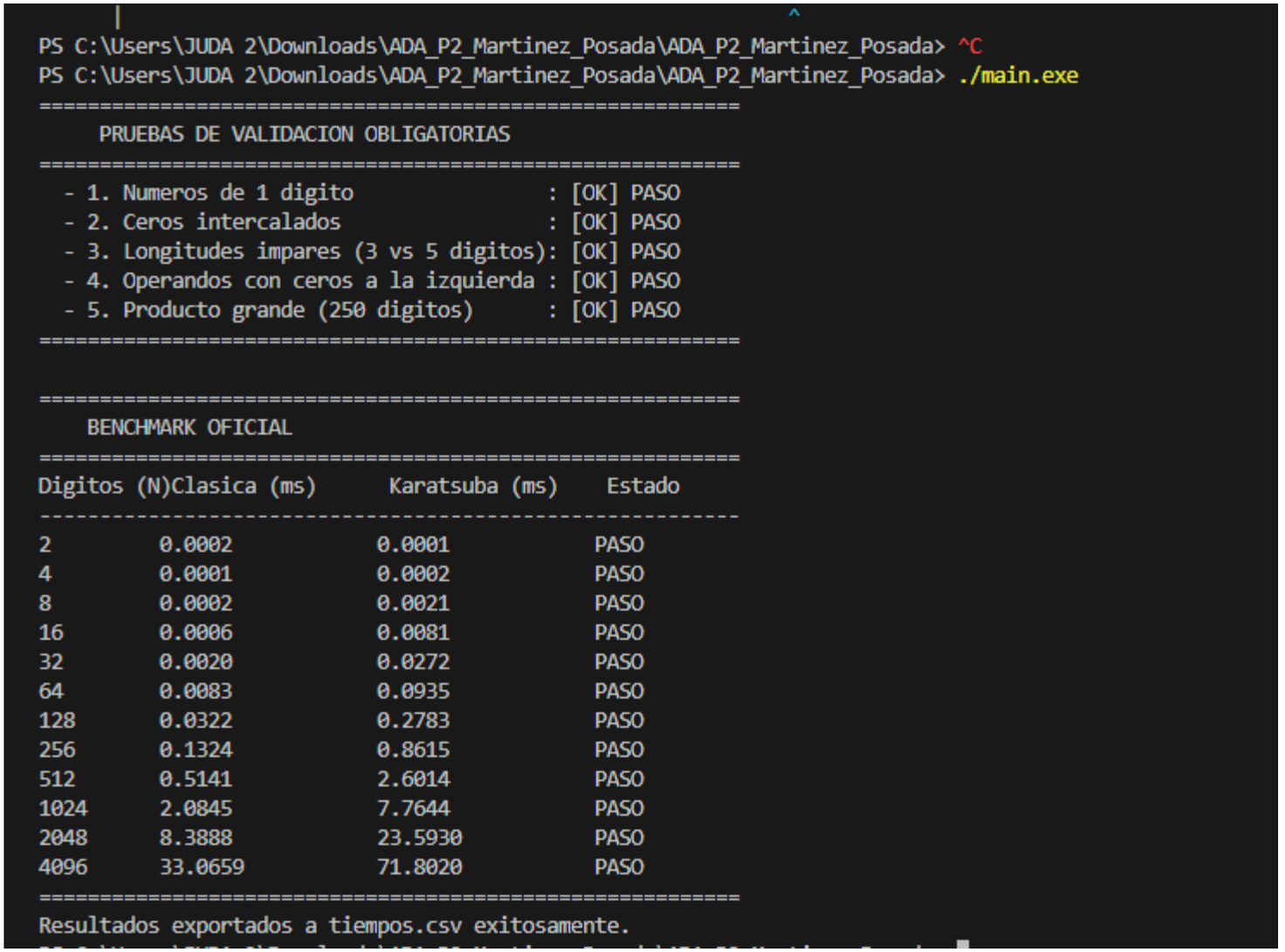
- **Representación de los números:** Decidimos utilizar std::vector<int> almacenando los números en formato Little-Endian. Esto facilitó enormemente las operaciones de suma y resta, ya que los desbordamientos (el "llevo una") fluían naturalmente hacia los índices mayores del vector, igual a como sumamos a mano en papel.
- **Relleno a potencias de 2 (Padding):** Dividir arreglos a la mitad suele generar problemas cuando la longitud es un número impar (un ejemplo es a partir un número de 7 dígitos). Para evitar errores de índices, implementamos un relleno automático: si los números no medían lo mismo, o si su tamaño no era potencia de 2, les agregábamos ceros a la izquierda (que en nuestro vector Little-Endian era simplemente agregar ceros al final del arreglo) hasta alcanzar la potencia de 2 más cercana. Esto garantizó particiones exactas en cada nivel de la recursión.
- **Prevención de resultados negativos:** En el cálculo del término central ($\mathbf{P_3 - P_1 - P_2}$). Al trabajar con vectores de dígitos enteros, si hacíamos esa resta de forma secuencial descuidada, podíamos obtener resultados parciales negativos, lo que obligaría a implementar una lógica compleja de "signos" para números muy grandes. Lo solucionamos ordenando la operación. Matemáticamente, P_3 (que incluye los productos cruzados más P_1 y P_2) siempre será más grande que P_1. Por lo tanto, calculamos primero $\mathbf{(P_3 - P_1)}$ y al resultado de eso le restamos P_2. Esto aseguró que todas las funciones auxiliares siempre manejaran arreglos de números positivos.

4. Análisis Experimental y Rendimiento

Para comprobar si la teoría se reflejaba en la práctica, implementamos un *benchmark* utilizando la librería std::chrono::steady_clock. Generamos

números aleatorios grandes y medimos el tiempo de ejecución en ms para ambos métodos.

Dígitos (N)	Tiempo Clásico (ms)	Tiempo Karatsuba (ms)	Estado de Validación
2	0.0002	0.0001	PASO
4	0.0001	0.0002	PASO
8	0.0002	0.0021	PASO
16	0.0006	0.0081	PASO
32	0.0020	0.0272	PASO
64	0.0083	0.0935	PASO
128	0.0322	0.2783	PASO
256	0.1324	0.8615	PASO
512	0.5141	2.6014	PASO
1024	2.0845	7.7644	PASO
2048	8.3888	23.5930	PASO
4096	33.0659	71.8020	PASO



```
PS C:\Users\JUDA 2\Downloads\ADA_P2_Martinez_Posada\ADA_P2_Martinez_Posada> ^C
PS C:\Users\JUDA 2\Downloads\ADA_P2_Martinez_Posada\ADA_P2_Martinez_Posada> ./main.exe
=========================================================
     PRUEBAS DE VALIDACION OBLIGATORIAS
=========================================================
  - 1. Numeros de 1 digito               : [OK] PASO
  - 2. Ceros intercalados                : [OK] PASO
  - 3. Longitudes impares (3 vs 5 digitos): [OK] PASO
  - 4. Operandos con ceros a la izquierda : [OK] PASO
  - 5. Producto grande (250 digitos)      : [OK] PASO
=========================================================

=========================================================
    BENCHMARK OFICIAL
=========================================================
Digitos (N)Clasica (ms)      Karatsuba (ms)    Estado
---------------------------------------------------------
2         0.0002             0.0001            PASO
4         0.0001             0.0002            PASO
8         0.0002             0.0021            PASO
16        0.0006             0.0081            PASO
32        0.0020             0.0272            PASO
64        0.0083             0.0935            PASO
128       0.0322             0.2783            PASO
256       0.1324             0.8615            PASO
512       0.5141             2.6014            PASO
1024      2.0845             7.7644            PASO
2048      8.3888             23.5930           PASO
4096      33.0659            71.8020           PASO
=========================================================
Resultados exportados a tiempos.csv exitosamente.
```

Comparativa de tiempos de ejecución (en ms) entre el algoritmo clásico y Karatsuba según el tamaño de la entrada.

Interpretación del Punto de Cruce

Al analizar los datos experimentales, nos encontramos con un fenómeno interesante: **en el rango evaluado (hasta 4096 dígitos), nunca se alcanzó el punto de cruce** donde Karatsuba superara sistemáticamente al algoritmo clásico. De hecho, para 4096 dígitos, el clásico tardó aproximadamente 33 milisegundos, mientras que Karatsuba requirió aproximadamente 72 milisegundos.

¿Por qué ocurre esto si la complejidad asintótica de Karatsuba es mejor? La respuesta radica en la arquitectura del hardware y las constantes ocultas de los lenguajes de programación. El algoritmo clásico está implementado con un doble bucle for que opera sobre un espacio de memoria contiguo previamente reservado. Para el procesador, esto es extremadamente fácil de predecir y ejecutar.

Por otro lado, nuestra implementación de Karatsuba depende de múltiples llamadas recursivas. En cada llamada se crean nuevos objetos std::vector, se redimensionan, se copian pedazos de arreglos (mitades altas y bajas) y se devuelve memoria al sistema. En C++, la asignación dinámica de memoria (heap allocation) es una operación muy lenta en comparación con una simple suma o multiplicación en el procesador. Para

números "pequeños", el sobrecosto de crear y destruir miles de vectores ahoga por completo el beneficio matemático de hacer menos multiplicaciones. Es muy probable que, con esta implementación específica en vectores, el punto de cruce real se encuentre en decenas de miles de dígitos, momento en el cual el crecimiento cuadrático del método clásico por fin se volvería más lento que la gestión de memoria de Karatsuba.

5. Declaración de uso ético de Inteligencia Artificial

Para el desarrollo integral de esta práctica, el equipo hizo uso de herramientas de Inteligencia Artificial estrictamente como un apoyo académico y de tutoría. Las consultas realizadas se enfocaron en la depuración de errores durante el manejo de los índices de los vectores en C++, en la estructuración de la explicación matemática del Teorema Maestro para asegurar claridad en la redacción, y en la generación de scripts para graficar los datos arrojados por nuestra terminal.

Afirmamos que el diseño algorítmico, la toma de decisiones para normalizar arreglos, el manejo de restas sin signos, y la comprensión del porqué el exponente es $\log_2(3)$, fueron analizados y asimilados completamente por los integrantes del grupo. La IA funcionó como un complemento para pulir el informe final, garantizando que el entendimiento propio sea el eje de este documento y de la posible sustentación individual.